Department of Computer Science

Faculty of Computing

Sri Lanka Institute of Information Technology

IT3021: Data Warehousing and Business Intelligence

Year 3 Semester 2 | 2026

Assignment 1

Electricity Transaction Data Warehouse & ETL Development

IT23245860

C A N Methma.

Table of Contents

TABLE OF CONTENTS **2**

1. INTRODUCTION **4**

2. DATASET SELECTION **5**

2.1 DATASET DESCRIPTION 5
2.2 BUSINESS PROCESS EXPLANATION 5
2.3 DATASET ATTRIBUTES 5
2.4 ENTITY RELATIONSHIP DIAGRAM (ERD) 7

3. PREPARATION OF DATA SOURCES **8**

3.1 OVERVIEW 8
3.2 DATA SOURCE MAPPING 8
3.3 DESCRIPTION OF EACH DATA SOURCE 8
Source 1 – Participants Core (CSV) *8*
Source 2 – Participants CRM (CSV) *8*
Source 3 – Network Nodes (Text File) *9*
Source 4 – Security Configurations (Database) *9*
Source 5 – Transactions (Text File) *9*
3.4 ENRICHMENT AND HIERARCHY PLANNING 9

4. SOLUTION ARCHITECTURE **10**

4.1 ARCHITECTURE OVERVIEW 10
4.2 ARCHITECTURE DIAGRAM 10
4.3 COMPONENT DESCRIPTIONS 11

5. DATA WAREHOUSE DESIGN & DEVELOPMENT **11**

5.1 DIMENSIONAL MODEL OVERVIEW 11
5.2 FACT TABLE – FACTTRANSACTIONS 11
5.3 DIMENSION TABLES 12
DimParticipant – SCD Type 2 Detail *12*
5.4 KEY DESIGN DECISION – DIMSECURITY NOT LINKED TO FACT 13
5.5 STAR SCHEMA DIAGRAM 14
5.6 SQL SERVER IMPLEMENTATION 15
5.7 DESIGN ASSUMPTIONS 16

6. ETL DEVELOPMENT **16**

6.1 OVERVIEW 16
6.2 ETL LOAD ORDER 17
6.3 SSIS CONTROL FLOW 18
6.4 STAGING – DATA EXTRACTION 18
6.5 LOADING DIMREGION 19
6.6 LOADING DIMNETWORK – SCD TYPE 1 19
6.7 LOADING DIMSECURITY 19
6.8 LOADING DIMPARTICIPANT – SCD TYPE 2 20
6.9 LOADING FACTTRANSACTIONS 22

6.10 ETL Execution and Validation 23
7. ETL DEVELOPMENT: ACCUMULATING FACT TABLE UPDATE 24
7.1 Concept 24
7.2 Accumulating Columns 24
7.3 Input Dataset – Completed Transactions 24
7.4 Update Package Architecture 25
7.5 Update Logic – OLE DB Command 25
7.6 Pre- and Post-Update Validation 26
8. Conclusion 27

1. Introduction

This report presents the design and implementation of a fully integrated Data Warehouse and Business Intelligence solution for analysing electricity transaction data. The system was developed using the Kimball dimensional modelling methodology, with Microsoft SQL Server as the database platform and SQL Server Integration Services (SSIS) as the ETL engine.

The solution addresses a real-world analytical challenge: consolidating electricity trading data from multiple heterogeneous operational sources into a unified, query-optimised data warehouse. The warehouse captures key business dimensions including market participants, network infrastructure, geographic regions, and security configurations, all structured around a central fact table recording individual electricity trade transactions.

The architecture follows industry-standard data warehousing principles, incorporating a staging layer for data cleansing and integration, a star schema dimensional model for analytical efficiency, Slowly Changing Dimension handling to preserve historical accuracy, and an accumulating fact table design to track the full lifecycle of each transaction from creation to completion.

The outcome is a robust analytical platform capable of supporting business intelligence queries across time, geography, network performance, and transaction cost — enabling data-driven decision-making within the electricity trading domain.

Data Set: Electricity Transaction Dataset

2. Dataset Selection

2.1 Dataset Description

The data set selected for this project is the Electricity Transaction Dataset, representing a transactional OLTP system used to record electricity trading activities between market participants. The system captures high-frequency trade transactions executed through a digital energy trading platform. It contains approximately 10,000 to 150,000 transaction records covering roughly one year of trading activity, which is sufficient to demonstrate all required data warehousing concepts.

Each transaction record captures:

- Transaction identifier and timestamp
- Participant details (user ID, role, account tier, company, region)
- Network infrastructure used (network slice, data centre, slice type)
- Security configuration applied (security level, encryption method, Zero Trust authentication)
- Financial measures (electricity quantity in MWh, price per MWh, total cost)
- Technical measure (network latency in milliseconds)
- Transaction metadata (type: buy/sell, status: completed/pending)

2.2 Business Process Explanation

The dataset models an electricity spot market trading process where energy generators, dealers, and consumers buy and sell electricity in real time through a digital platform. When a participant initiates a trade, the system records the full transaction event including financial, network, and security details.

Each transaction is associated with three key operational entities:

- A market participant (the user executing the trade)
- A network node (the infrastructure slice used to transmit the trade)
- A security configuration (the encryption and authentication settings active during the trade)

This design ensures traceability, operational auditability, and supports rich analytical querying across time, geography, network performance, and transaction cost dimensions.

2.3 Dataset Attributes

Attribute	Description
TransactionID	Unique identifier for each electricity trade transaction
Timestamp	Date and time when the transaction occurred
UserID	Unique identifier of the market participant
UserRole	Role of the participant (generator, dealer, consumer)

AccountTier	Account classification tier of the participant
CompanyName	Company the participant belongs to
RegionName	Geographic region of the participant
NetworkSliceID	Identifier of the network slice used
DataCenter	Data centre handling the transaction
SliceType	Type of network slice (e.g., eMBB, URLLC)
SecurityConfigID	Identifier of the security configuration applied
SecurityLevel	Level of security applied to the transaction
EncryptionMethod	Encryption algorithm used (e.g., AES-256)
ZTAuthentication	Whether Zero Trust Authentication was applied (Y/N)
TransactionType	Trade direction: Buy or Sell
ElectricityQuantity	Amount of electricity traded (MWh)
PricePerMWh	Price per megawatt-hour (USD)
TotalCost	Total monetary value of the transaction
LatencyMS	Network latency in milliseconds
TransactionStatus	Outcome of the transaction (Completed / Pending)

2.4 Entity Relationship Diagram (ERD)

The ER diagram below illustrates the OLTP structure of the electricity trading system. The central Transaction entity links to Participant, Network Node, and Security Configuration entities through foreign key relationships.

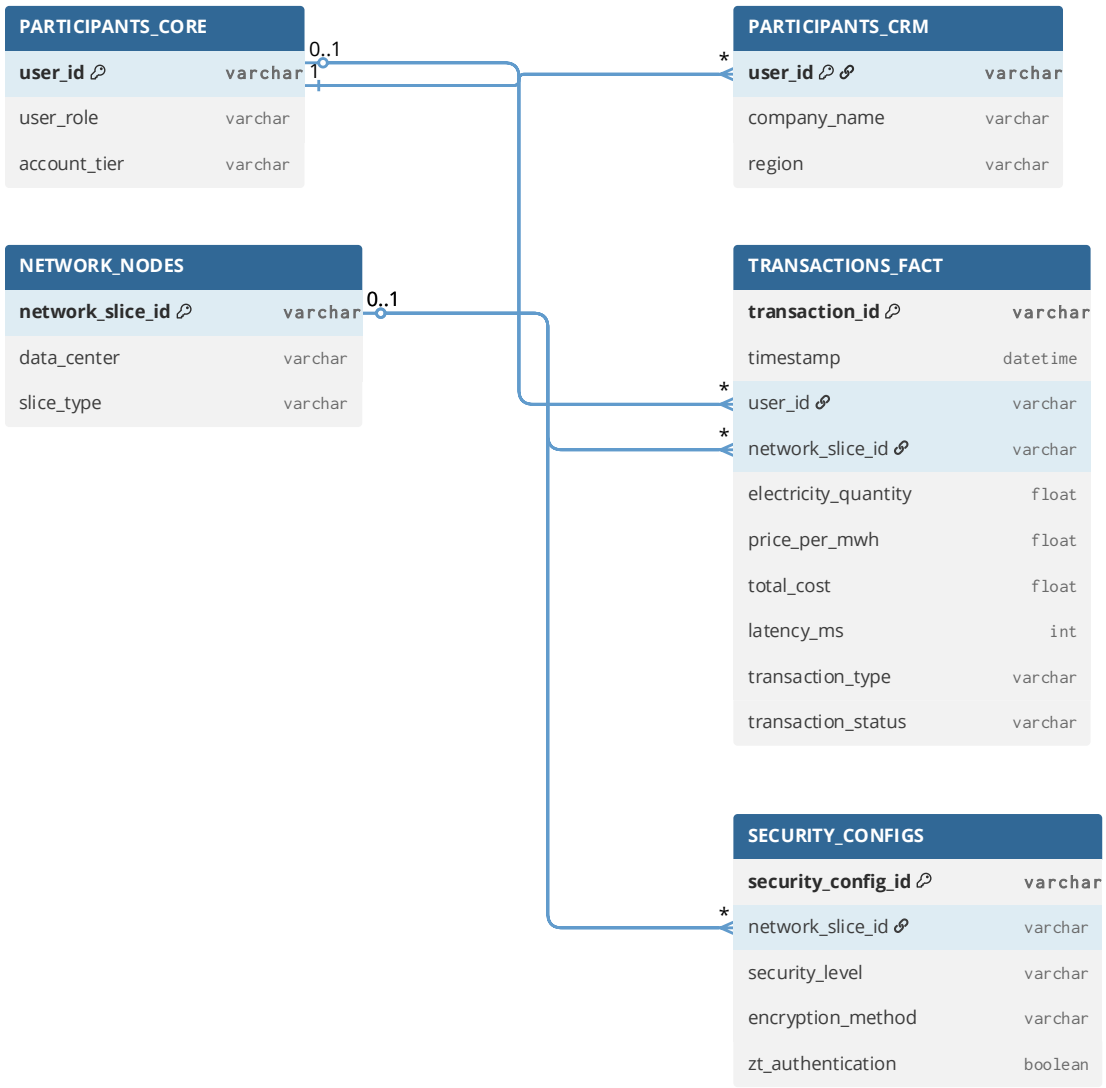

Figure 1 – OLTP Entity Relationship Diagram (ERD)

3. Preparation of Data Sources

3.1 Overview

In a real-world electricity trading environment, operational data is not stored in a single system. Different aspects of the business are managed by separate systems, participant information is maintained by the CRM and HR teams, network infrastructure data comes from the network operations team, security configurations are managed by the IT security department, and transaction records are generated by the trading platform itself.

To reflect this realistic scenario, the data for this project is sourced from five distinct files across multiple formats, representing three separate operational systems. These sources were loaded into five staging tables in the data warehouse pipeline.

3.2 Data Source Mapping

Staging Table	Source File / Format	Contents
stg_Participants_Core	CSV file	UserID, UserRole, AccountTier — core participant identity data
stg_Participants_CRM	CSV file	UserID, CompanyName, RegionName — CRM and geographic data
stg_Network_Nodes	Text file	NetworkSliceID, DataCenter, SliceType — network infrastructure records
stg_Security_Configs	SQL Server Database table	SecurityConfigID, SecurityLevel, EncryptionMethod, ZTAuthentication — security settings
stg_Transactions_Fact	Text File	All transaction records — the primary OLTP transactional source

3.3 Description of Each Data Source

Source 1 – Participants Core (CSV)

This CSV file contains the core identity attributes of each market participant.

- UserID — business key linking to CRM source
- UserRole — generator, dealer, or consumer
- AccountTier — classification of the participant account

Source 2 – Participants CRM (CSV)

This CSV file represents a CRM system holding company and geographic data for participants. It is merged with the Core source during ETL using a Merge Join on UserID to produce the complete DimParticipant dimension.

- UserID — join key to Core source
- CompanyName — name of the participant's organisation
- RegionName — geographic region of the participant

Source 3 – Network Nodes (Text File)

This text file represents network infrastructure records describing the communication slices used during transactions.

- NetworkSliceID — unique identifier for the network slice
- DataCenter — data centre handling network traffic
- SliceType — type of network slice (e.g., eMBB, URLLC)

Source 4 – Security Configurations (Database)

This database table stores security configurations applied during transactions. Storing security settings in a relational database reflects a real-world scenario where a dedicated security management system maintains these configurations.

- SecurityConfigID — unique configuration identifier
- SecurityLevel — security classification level
- EncryptionMethod — encryption algorithm applied
- ZTAuthentication — Zero Trust Authentication flag (Y/N)

Source 5 – Transactions (Text File)

This text file is the primary OLTP transactional source containing all electricity trade event records. It holds the core financial and operational measures along with foreign key references to participants, network nodes, and security configurations.

3.4 Enrichment and Hierarchy Planning

The separation of participants into two sources (Core + CRM) introduces the RegionName attribute, which is used to build the DimRegion dimension. This enables geographic hierarchical analysis (Region → Participant) that would not be possible from the transaction data alone.

The Network source introduces SliceType and DataCenter, enabling network-level analysis and grouping. These enrichments allow the warehouse to support richer analytical queries beyond what the raw transaction data provides.

4. Solution Architecture

4.1 Architecture Overview

The DW & BI solution follows a layered architecture that moves data from raw operational sources through to analytical reporting tools.

4.2 Architecture Diagram

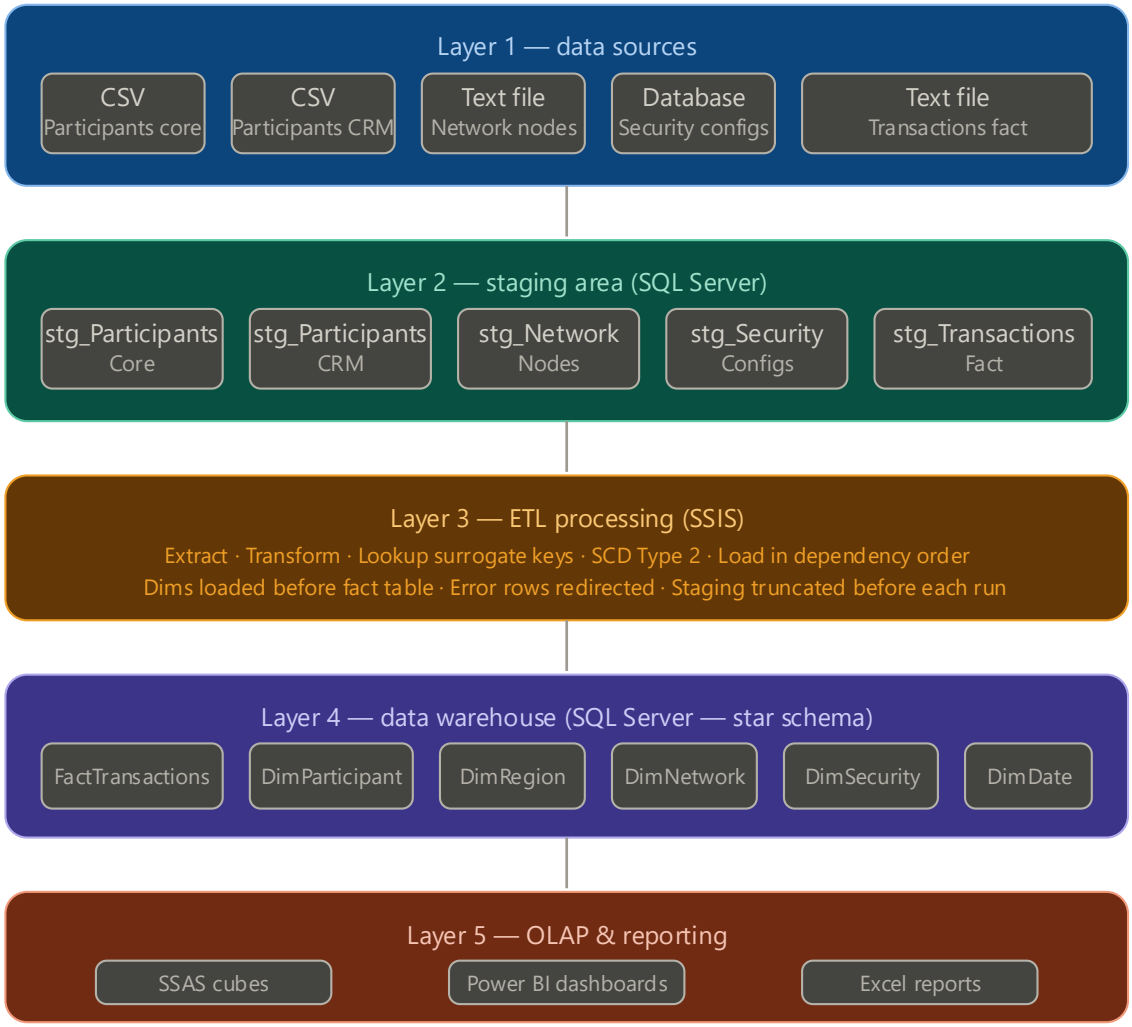

Figure 2 – Data Warehouse & BI Solution Architecture

4.3 Component Descriptions

Layer	Components	Description
Data Source Layer	CSV, Text files, SQL Server Database	Operational sources holding raw participant, network, security, and transaction data in heterogeneous formats
Staging Layer	stg_Participants_Core, stg_Participants_CRM, stg_Network_Nodes, stg_Security_Configs, stg_Transactions_Fact	Temporary SQL Server tables that hold extracted raw data before transformation. Enables data cleansing, schema alignment, and isolation from production source systems
ETL Layer	SSIS (SQL Server Integration Services)	SSIS packages extract data from all sources, apply transformations (Merge Join, Lookup, SCD, Derived Column), resolve surrogate keys, and load data into the dimensional model in the correct order
Data Warehouse Layer	SQL Server DW Database	Central analytical repository implementing a Star Schema with one fact table and five dimension tables, optimized for query performance
OLAP & Reporting Layer	SSAS, Power BI, Excel	Business intelligence tools that connect to the DW to generate reports, dashboards, and multidimensional analysis cubes

5. Data Warehouse Design & Development

5.1 Dimensional Model Overview

The data warehouse is designed using a Star Schema following the Kimball dimensional modelling methodology. The central fact table FactTransactions is surrounded by five dimension tables, all connected directly to the fact table via surrogate keys.

5.2 Fact Table – FactTransactions

One row in FactTransactions represents one electricity trade transaction executed by a market participant through the trading platform.

The fact table contains three categories of columns:

Category	Column	Description
Foreign Key	ParticipantSK	Surrogate key linking to DimParticipant
Foreign Key	NetworkSK	Surrogate key linking to DimNetwork
Foreign Key	DateKey	Surrogate key linking to DimDate
Measure	ElectricityQuantity	Amount of electricity traded (MWh)

Measure	PricePerMWh	Price per megawatt-hour (USD)
Measure	TotalCost	Total monetary value of the transaction
Measure	LatencyMS	Network latency in milliseconds
Degenerate Dimension	TransactionID	Natural business key — retained in fact, no dimension table
Degenerate Dimension	TransactionType	Buy or Sell — low cardinality attribute stored directly
Degenerate Dimension	TransactionStatus	Completed or Pending
Accumulating	accm_txn_create_time	Timestamp when the record was inserted into the DW
Accumulating	accm_txn_complete_time	Timestamp when the transaction was confirmed complete (initially NULL)
Accumulating	txn_process_time_hours	Computed duration in hours between create and complete time

5.3 Dimension Tables

Dimension	Key	Attributes	SCD Type
DimParticipant	ParticipantSK	UserID, UserRole, AccountTier, CompanyName, RegionSK, StartDate, EndDate, IsCurrent	Type 2
DimRegion	RegionSK	RegionName	Type 1
DimNetwork	NetworkSK	NetworkSliceID, DataCenter, SliceType	Type 1
DimSecurity	SecurityConfigSK	SecurityConfigID, SecurityLevel, EncryptionMethod, ZTAuthentication	Type 1 (not linked to fact — see design decision)
DimDate	DateKey	Date, Day, Month, Quarter, Year, DayOfWeek, IsWeekend	Static (pre-populated)

DimParticipant – SCD Type 2 Detail

The DimParticipant dimension is implemented as a Type 2 Slowly Changing Dimension to track historical changes in participant attributes such as UserRole and AccountTier. When a change is detected, the existing record is expired and a new record is inserted.

SCD Column	Purpose
StartDate	Date from which this version of the participant record became effective
EndDate	Date on which this version expired (NULL for the currently active record)
IsCurrent	Flag indicating the active record: 1 = current, 0 = historical

5.4 Key Design Decision – DimSecurity Not Linked to Fact

DimSecurity is populated and maintained as a dimension table, but it is deliberately not linked to the FactTransactions fact table via a foreign key. This is an intentional design decision based on the following reasoning:

- Security configuration is associated with the network infrastructure, not with individual transactions at the grain level.
- Linking DimSecurity directly to the fact would introduce ambiguity, as multiple security configurations may be associated with a single network node.
- Including a SecuritySK in the fact table could produce duplicate fact rows or incorrect aggregations when analysing measures such as TotalCost or ElectricityQuantity.

Security-level analysis can still be performed by joining DimSecurity to DimNetwork (via NetworkSK) in analytical queries — without compromising fact table integrity.

5.5 Star Schema Diagram

The diagram below shows the complete Star Schema implemented in SQL Server, with FactTransactions at the centre surrounded by all dimension tables.

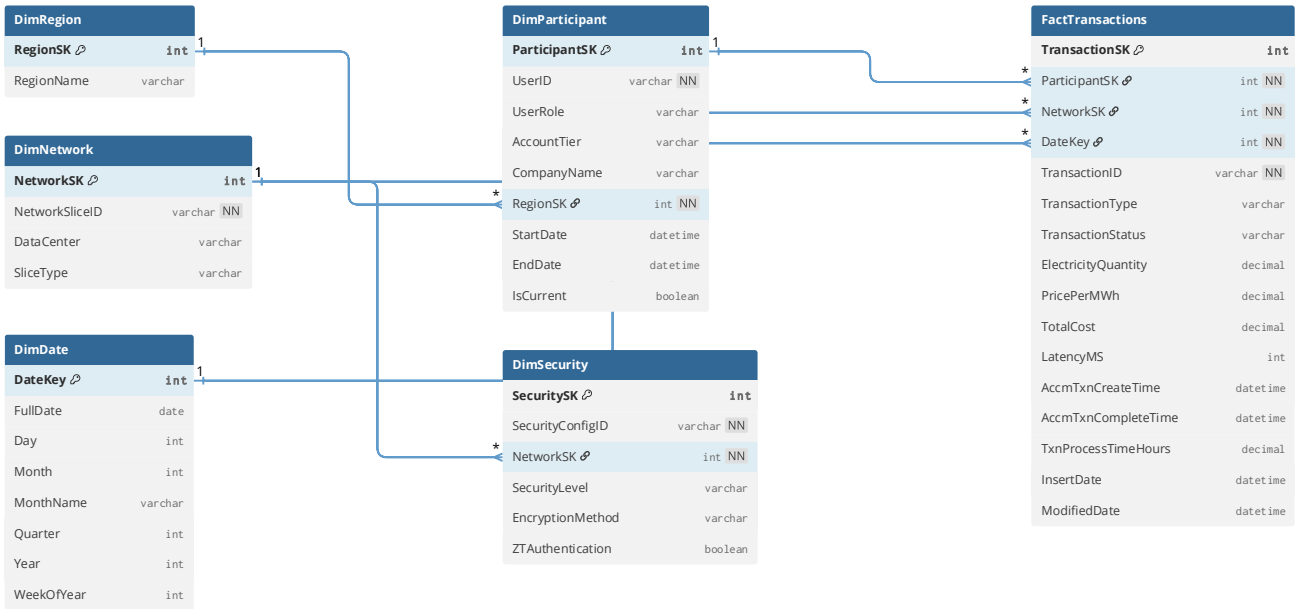

Figure 3 – Data Warehouse Star Schema

5.6 SQL Server Implementation

The dimensional model was implemented in SQL Server using CREATE TABLE statements. Below are the key table definitions.

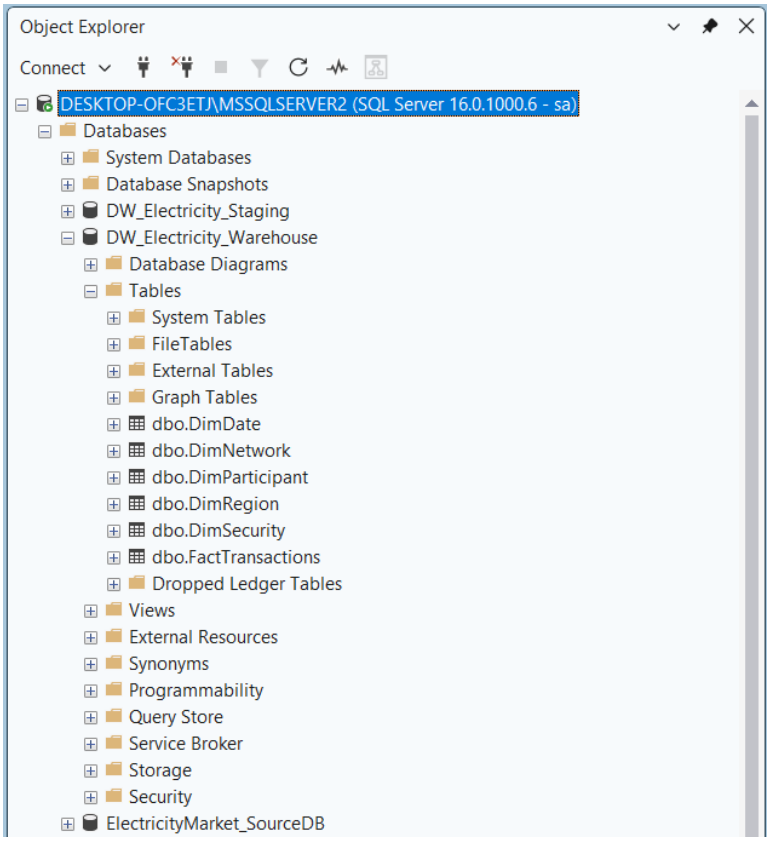

Figure 4 – DW Tables in SQL Server Management Studio

FactTransactions – SQL Definition:

```
CREATE TABLE FactTransactions (
    TransactionSK          INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantSK          INT NOT NULL,
    NetworkSK              INT NOT NULL,
    DateKey                INT NOT NULL,
    TransactionID          NVARCHAR(50),      -- degenerate dimension
    TransactionType        NVARCHAR(10),      -- degenerate dimension
    TransactionStatus      NVARCHAR(20),      -- degenerate dimension
    ElectricityQuantity    DECIMAL(10,2),
    PricePerMWh            DECIMAL(10,2),
    TotalCost              DECIMAL(12,2),
    LatencyMS              INT,
    accm_txn_create_time   DATETIME DEFAULT GETDATE(),
    accm_txn_complete_time DATETIME NULL,
    txn_process_time_hours FLOAT NULL,
```

```
    FOREIGN KEY (ParticipantSK) REFERENCES DimParticipant(ParticipantSK),
    FOREIGN KEY (NetworkSK)     REFERENCES DimNetwork(NetworkSK),
    FOREIGN KEY (DateKey)       REFERENCES DimDate(DateKey)
);
```

DimParticipant – SQL Definition (SCD Type 2):

```
CREATE TABLE DimParticipant (
    ParticipantSK  INT IDENTITY(1,1) PRIMARY KEY,
    UserID         NVARCHAR(50)    NOT NULL,   -- business key
    UserRole       NVARCHAR(50),
    AccountTier    NVARCHAR(50),
    CompanyName    NVARCHAR(100),
    RegionSK       INT,
    StartDate      DATE            NOT NULL,
    EndDate        DATE            NULL,
    IsCurrent      BIT             NOT NULL DEFAULT 1
);
```

5.7 Design Assumptions

1. TransactionID is treated as the natural business key and stored as a degenerate dimension in the fact table.
2. Surrogate keys are generated using SQL Server IDENTITY columns for all dimension and fact tables.
3. DimParticipant implements SCD Type 2; all other dimensions use SCD Type 1 (update in place).
4. DimDate is pre-populated to cover the full date range of the dataset before ETL execution.
5. DimSecurity is intentionally excluded from the fact table foreign keys — it is linked analytically via DimNetwork.
6. Null or unmatched records during Lookup transformations are redirected to error outputs rather than causing ETL failure.
7. The grain of the fact table is one row per transaction — no pre-aggregation is applied during ETL.

6. ETL Development

6.1 Overview

The ETL pipeline was implemented using SQL Server Integration Services (SSIS) within Microsoft Visual Studio. The pipeline is organised into two SSIS packages: one for staging data extraction and one for dimension and fact loading. Data flows from source files through staging tables into the dimensional model.

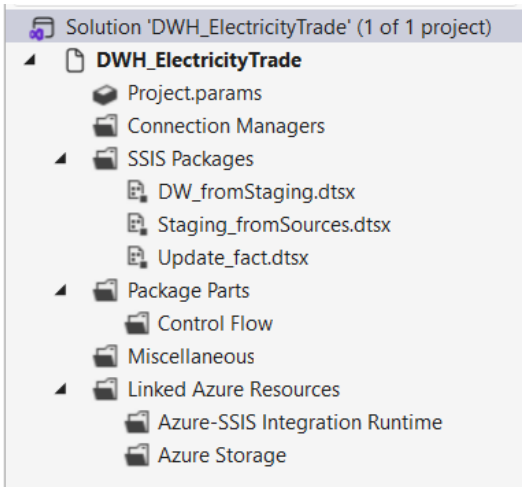

Figure 5.1 – SSIS Project Structure

6.2 ETL Load Order

Referential integrity requires that all dimension tables are fully loaded before the fact table is populated. The execution sequence is:

Step	Task	Target
1	Truncate all staging tables	All stg_ tables
2	Extract from CSV → stg_Participants_Core	stg_Participants_Core
3	Extract from CSV → stg_Participants_CRM	stg_Participants_CRM
4	Extract from Text → stg_Network_Nodes	stg_Network_Nodes
5	Extract from Database → stg_Security_Configs	stg_Security_Configs
6	Extract from Text → stg_Transactions_Fact	stg_Transactions_Fact
7	Load DimRegion	DimRegion
8	Load DimNetwork (Type 1 SCD)	DimNetwork
9	Load DimSecurity (Lookup NetworkSK)	DimSecurity
10	Load DimParticipant (Type 2 SCD)	DimParticipant
11	Load DimDate (if not pre-populated)	DimDate
12	Load FactTransactions	FactTransactions

6.3 SSIS Control Flow

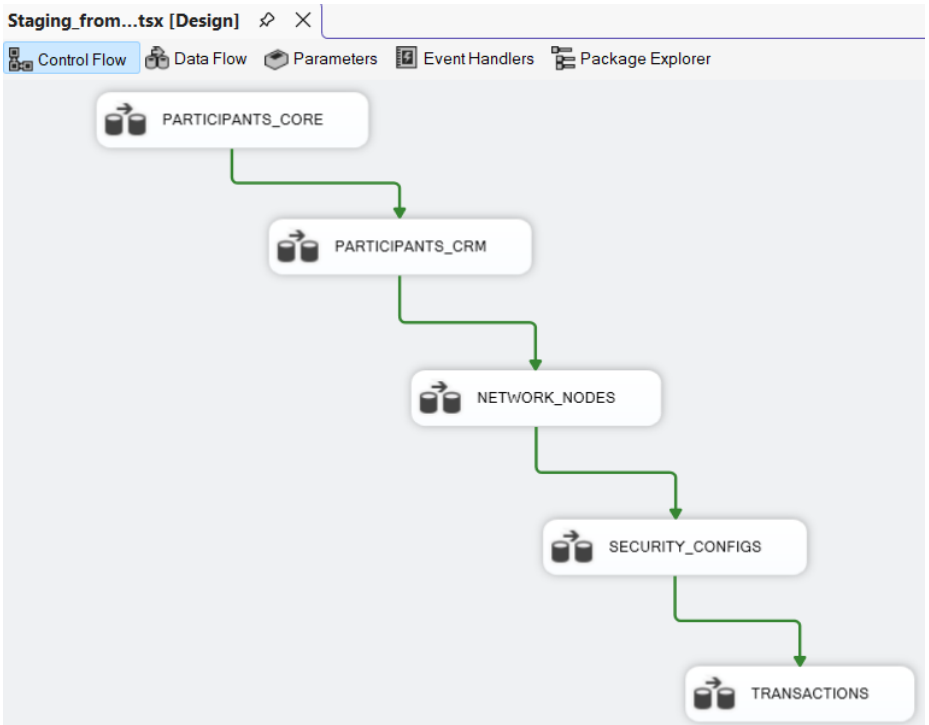

Figure 5.2 – Core ETL Package Control Flow

6.4 Staging – Data Extraction

Each staging table is truncated before extraction using an Execute SQL Task to ensure only the latest data is processed on every ETL run. Data is then extracted using source components appropriate to each file format:

Source	Format	SSIS Source Component	Staging Table
Participants_Core.csv	CSV	Flat File Source	stg_Participants_Core
Participants_CRM.csv	CSV	Flat File Source	stg_Participants_CRM
Network_Nodes.txt	CSV	Flat File Source	stg_Network_Nodes
Security_Configs (DB table)	Database	OLE DB Source	stg_Security_Configs
Transactions_Fact.txt	Text	Flat File Source	stg_Transactions_Fact

Figure 5.3 – Staging Extraction Data Flow Task

6.5 Loading DimRegion

Region data is derived from the RegionName column in stg_Participants_CRM. Duplicates are removed using a Sort and Aggregate transformation before inserting unique region records into DimRegion.

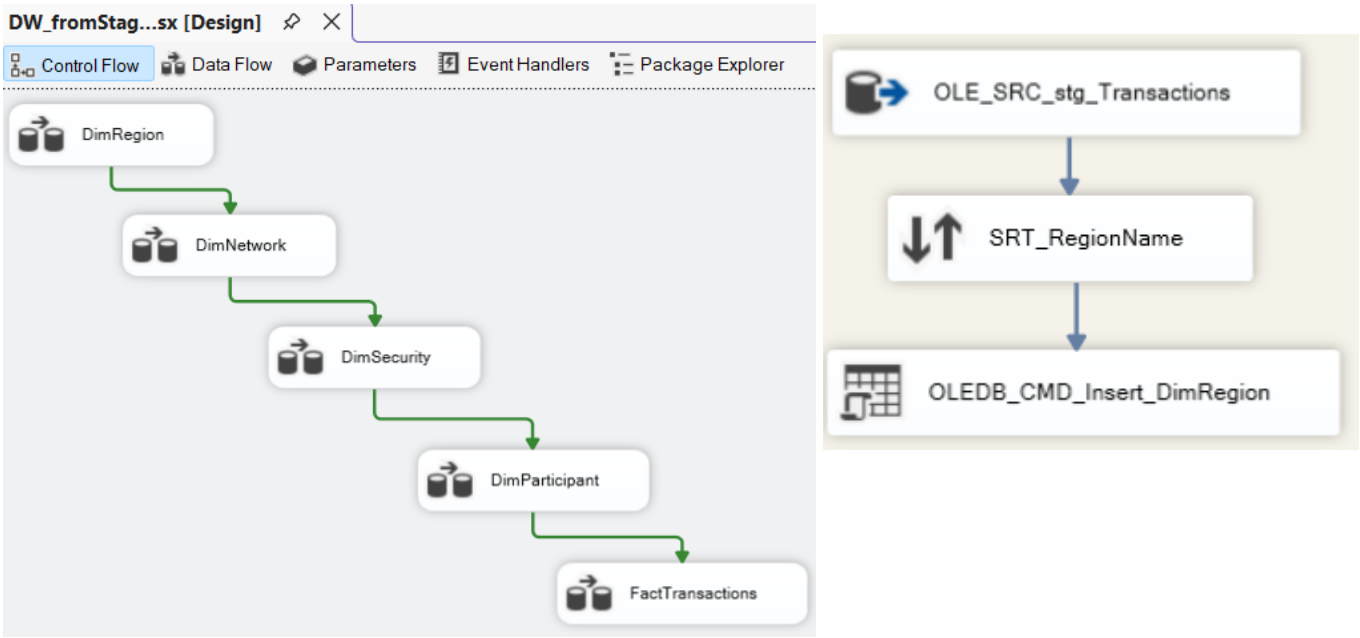

Figure 5.4 – DimRegion Load Data Flow

6.6 Loading DimNetwork – SCD Type 1

Network node records from stg_Network_Nodes are loaded into DimNetwork using a Lookup to check for existing records. New records are inserted; existing records with changed attributes are updated in place (Type 1 — no history preserved).

Figure 5.5 – DimNetwork Load Data Flow (SCD Type 1)

6.7 Loading DimSecurity

Security configuration records are extracted from the stg_Security_Configs database table using an OLE DB Source. A Lookup transformation resolves the NetworkSK from DimNetwork before inserting into DimSecurity.

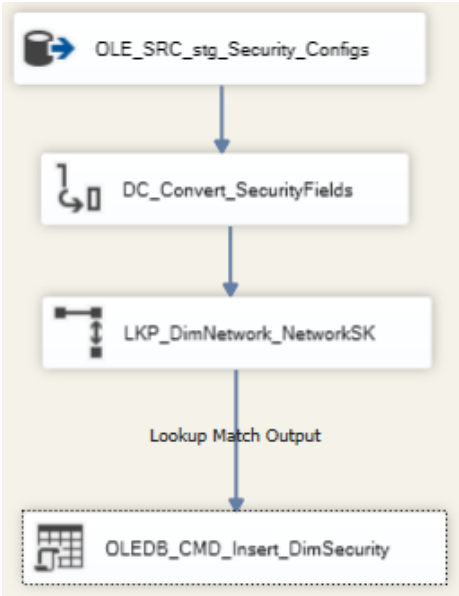

Figure 5.6 – DimSecurity Load Data Flow

8. SCD Configuration	9. Value
10. Business Key	11. UserID
12. Historical Attributes (Type 2)	13. UserRole, AccountTier
14. Fixed Attributes	15. CompanyName, RegionSK
16. StartDate Column	17. StartDate
18. EndDate Column	19. EndDate
20. IsCurrent Column	21. IsCurrent

6.8 Loading DimParticipant – SCD Type 2

In DimParticipant, combining data from two source tables and implementing Type 2 SCD to preserve historical participant records.

Steps in the Data Flow:

22. Sort stg_Participants_Core on UserID
23. Sort stg_Participants_CRM on UserID
24. Merge Join (Inner Join on UserID) to combine role/tier data with company/region data
11. Lookup DimRegion on RegionName → resolve RegionSK
12. Apply SCD Transformation (business key: UserID; historical attributes: UserRole, AccountTier)
13. On change detected: expire old record (update EndDate, set IsCurrent = 0) and insert new record

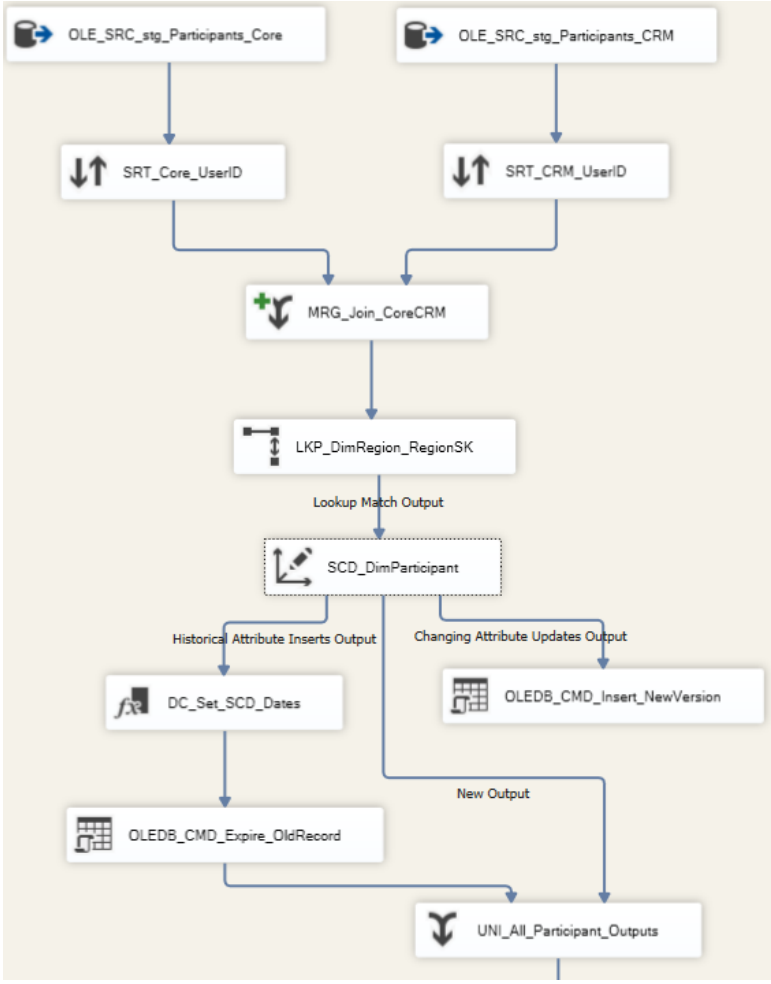

Figure 5.7 – DimParticipant Load Data Flow (SCD Type 2)

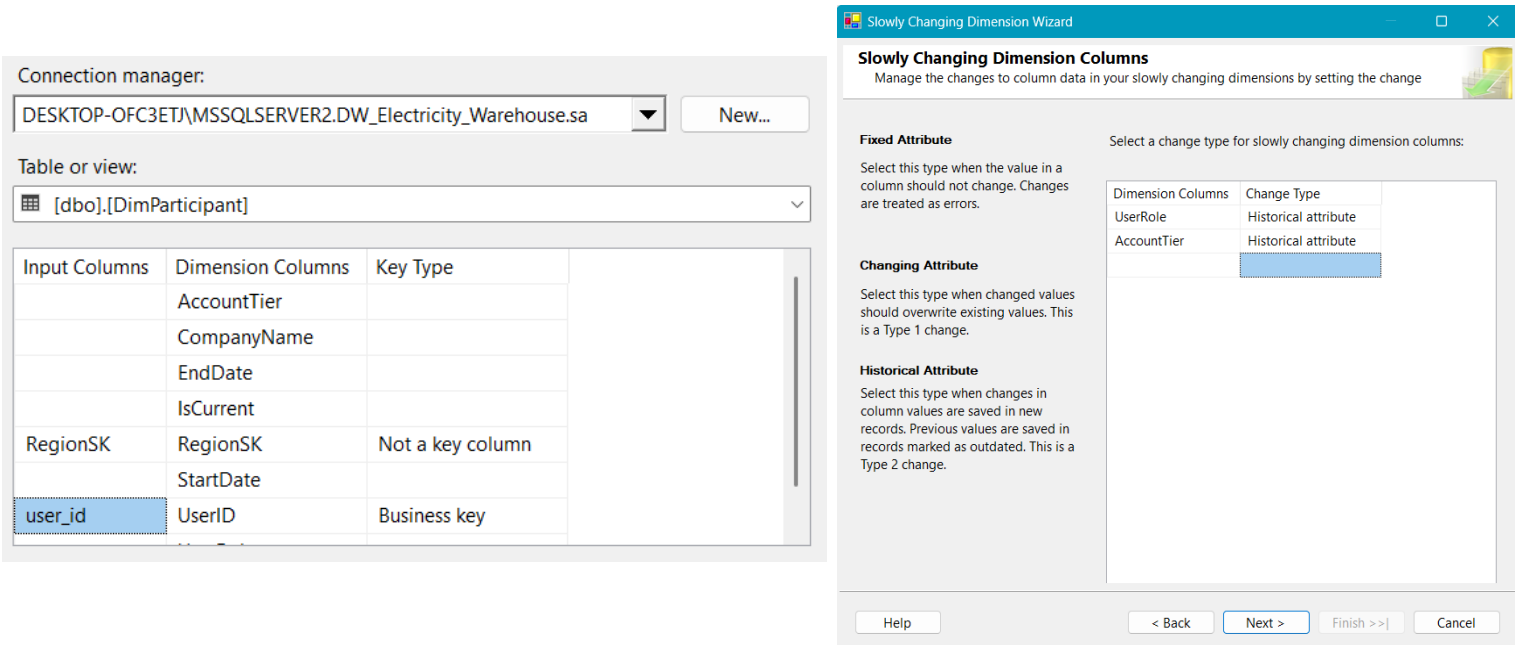

Figure 5.8 – SCD Type 2 Wizard Configuration

6.9 Loading FactTransactions

The fact table is loaded last, after all dimensions are fully populated. Surrogate keys are resolved using Lookup transformations for each dimension.

Steps in the Data Flow:

1. Flat File Source from stg_Transactions_Fact (Text file)
2. Derived Column: extract DateKey from transaction timestamp
3. Lookup DimParticipant on UserID → resolve ParticipantSK (IsCurrent = 1)
4. Lookup DimNetwork on NetworkSliceID → resolve NetworkSK
5. Lookup DimDate on date value → resolve DateKey
6. Derived Column: set accm_txn_create_time = GETDATE()
7. OLE DB Destination: insert into FactTransactions

Lookup	Match Key	Output Surrogate Key
DimParticipant	UserID (where IsCurrent = 1)	ParticipantSK
DimNetwork	NetworkSliceID	NetworkSK
DimDate	Derived date from timestamp	DateKey

All Lookup transformations are configured to redirect non-matching rows to an error output rather than failing the pipeline, ensuring fault tolerance.

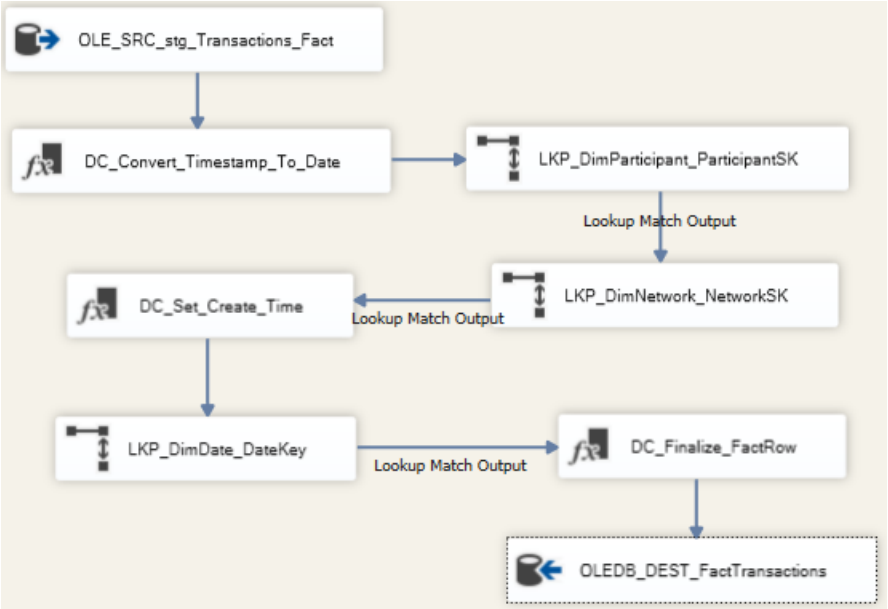

Figure 5.9 – FactTransactions Load Data Flow

6.10 ETL Execution and Validation

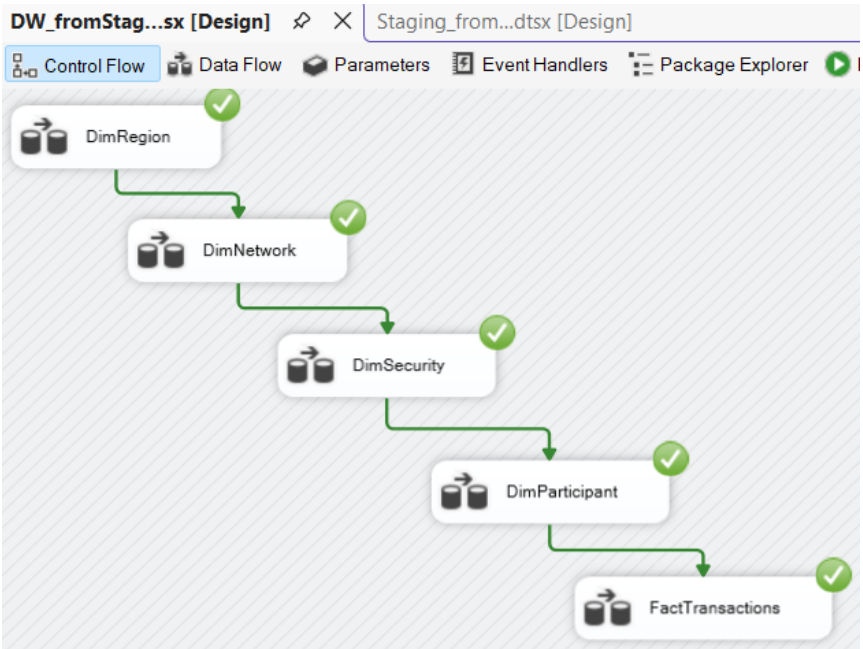

Figure 5.10 – Successful ETL Package Execution

Post-execution validation queries:

```
-- Total transactions loaded
SELECT COUNT(*) AS TotalTransactions FROM FactTransactions;
```

```
-- NULL surrogate key check
SELECT * FROM FactTransactions WHERE ParticipantSK IS NULL;
```

```
-- SCD Type 2 verification
SELECT UserID, COUNT(*) AS Versions
FROM DimParticipant GROUP BY UserID HAVING COUNT(*) > 1;
```

	TotalTransactions
1	26000

TransactionSK	ParticipantSK	NetworkSK	DateKey	TransactionID	ElectricityQuantity	PricePerMWh	TotalCost	LatencyMS	TransactionType	TransactionStatus	InsertDate

	UserID	Versions
1	U7536	3
2	U9813	2
3	U1023	2
4	U2794	3
5	U9244	2
6	U2395	2
7	U4997	2
8	U7069	5
9	U7426	3
10	U5949	2
11	U8292	2
12	U1613	5
13	U3485	3

Figure 5.11 – ETL Validation Query Results in SSMS

7. ETL Development: Accumulating Fact Table Update

7.1 Concept

An Accumulating Fact Table tracks the full lifecycle of a business process by allowing selected columns to be updated as the process moves through defined milestones. Unlike insert-only fact tables, accumulating fact tables support updates to reflect progress.

In this system, each electricity transaction passes through the following lifecycle stages:

8. Transaction Created — recorded when the fact record is first inserted into the DW
9. Transaction Processing — ongoing, no update needed
10. Transaction Completed — recorded when the trading platform confirms the transaction is finalised

7.2 Accumulating Columns

Column	Type	Description
accm_txn_create_time	DATETIME	Set to GETDATE() at initial fact load. Represents when the transaction entered the DW.
accm_txn_complete_time	DATETIME NULL	NULL at initial load. Updated when transaction completion is confirmed.
txn_process_time_hours	FLOAT NULL	Calculated as DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time) on update.

7.3 Input Dataset – Completed Transactions

A separate CSV file (Completed_Transactions.csv) was prepared to simulate the completion event feed. Its structure is:

Column	Description
TransactionSK	Surrogate key matching the record in FactTransactions to be updated
accm_txn_complete_time	The confirmed completion timestamp for the transaction

7.4 Update Package Architecture

A dedicated SSIS package named Update_Accumulating_Fact.dtsx was developed separately from the main ETL package to handle transaction lifecycle updates. This separation ensures that completion updates are processed independently without interfering with the main load pipeline.

7.5 Update Logic – OLE DB Command

Inside the update package, a Flat File Source reads Completed_Transactions.csv. An OLE DB Command transformation then performs a row-by-row parameterised UPDATE on the fact table.

SQL Update Statement used in OLE DB Command:

```
UPDATE FactTransactions
SET
   accm_txn_complete_time  = ?,
   txn_process_time_hours  = DATEDIFF(HOUR, accm_txn_create_time, ?)
WHERE
   TransactionSK          = ?
   AND accm_txn_complete_time IS NULL
```

Parameter Mapping:

Parameter	Mapped Input Column	Purpose
Param_0 (?)	accm_txn_complete_time	Sets the completion timestamp
Param_1 (?)	accm_txn_complete_time	Used in DATEDIFF to calculate hours
Param_2 (?)	TransactionSK	Matches the fact row to update

The condition AND accm_txn_complete_time IS NULL ensures that already-completed transactions are never overwritten. If a TransactionSK in the input file does not exist in the fact table, the UPDATE silently affects zero rows — no error is raised and the pipeline continues.

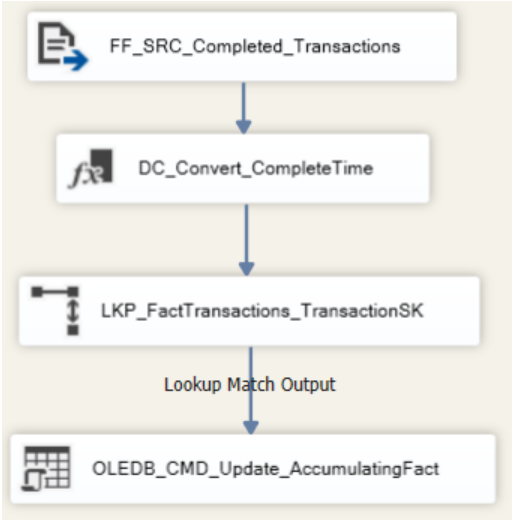

Figure 6.2 – Update Package Data Flow

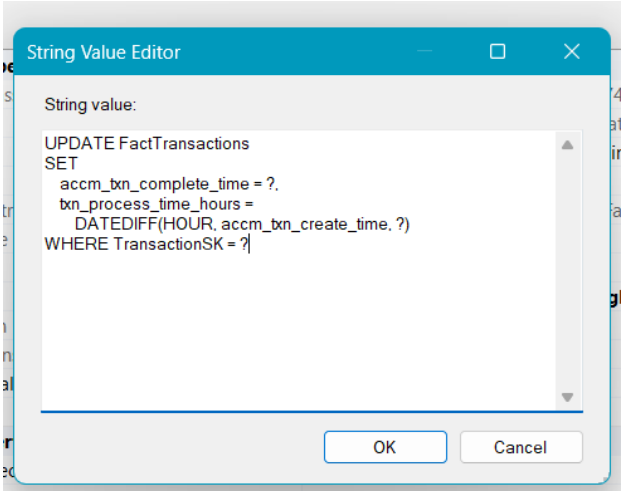

Figure 6.3 – OLE DB Command SQL Update Logic

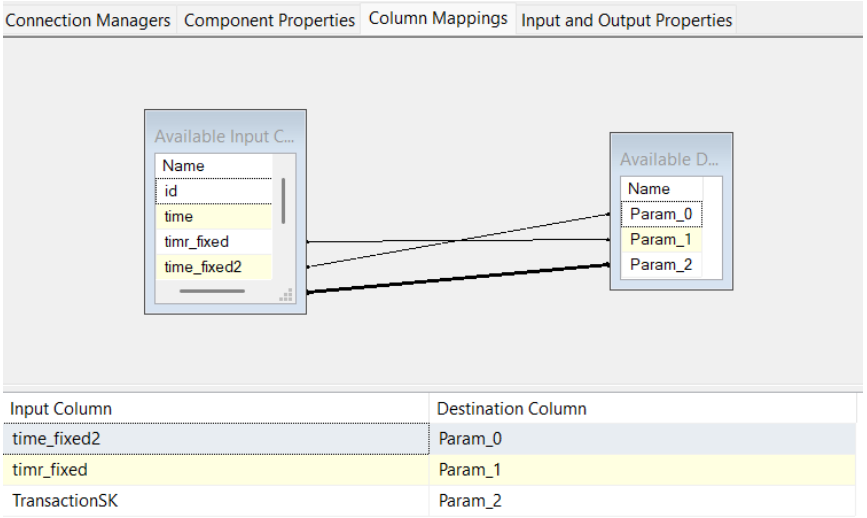

Figure 6.4 – Parameter Mapping Configuration

7.6 Pre- and Post-Update Validation

Validation queries were run in SSMS before and after executing the update package to confirm correct behaviour.

Pre-Update Query:

```
SELECT TransactionSK, accm_txn_create_time,
    accm_txn_complete_time, txn_process_time_hours
FROM FactTransactions
WHERE TransactionSK IN (101, 102, 103);
```

	TransactionSK	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	101	2026-03-28 18:19:04.827	NULL	NULL
2	102	2026-03-28 18:19:04.827	NULL	NULL
3	103	2026-03-28 18:19:04.827	NULL	NULL

Figure 6.5 – Fact Table Before Update

Post-Update Query (same query re-run after package execution):

	TransactionSK	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	10	2026-03-28 18:19:04.827	2026-03-29 10:19:04.827	16
2	12	2026-03-28 18:19:04.827	2026-03-30 13:19:04.827	43
3	13	2026-03-28 18:19:04.827	2026-03-30 05:19:04.827	35

Figure 6.6 – Fact Table After Update

8. Conclusion

This report documented the complete design and implementation of a Data Warehouse and Business Intelligence solution for Electricity Transaction Analysis using the Kimball dimensional modelling methodology.

The solution successfully delivers:

- A novel OLTP dataset representing real-time electricity trading activities, covering approximately one year of transaction data.
- Five data sources across three distinct formats (CSV, Text file, SQL Server Database) mapped to staging tables, exceeding the minimum requirement of two source types.
- A five-layer DW & BI architecture covering source ingestion, staging, ETL, dimensional storage, and analytical reporting.
- A Star Schema dimensional model implemented in SQL Server with one fact table (FactTransactions) and five dimension tables (DimParticipant, DimRegion, DimNetwork, DimSecurity, DimDate).
- SCD Type 2 on DimParticipant to track historical changes in participant roles and account tiers; SCD Type 1 on DimNetwork.
- A full SSIS ETL pipeline with multi-source extraction, Merge Join for participant consolidation, surrogate key lookups, SCD processing, and fact table loading in the correct dependency order.
- An Accumulating Fact Table implementation with a dedicated SSIS update package that tracks transaction lifecycle milestones and computes processing duration in hours.
- A deliberate design decision to exclude DimSecurity from the fact table to preserve grain integrity and prevent incorrect aggregations.
- All components were validated using SSMS queries confirming correct row counts, surrogate key assignments, SCD versioning, and accurate accumulating fact updates.